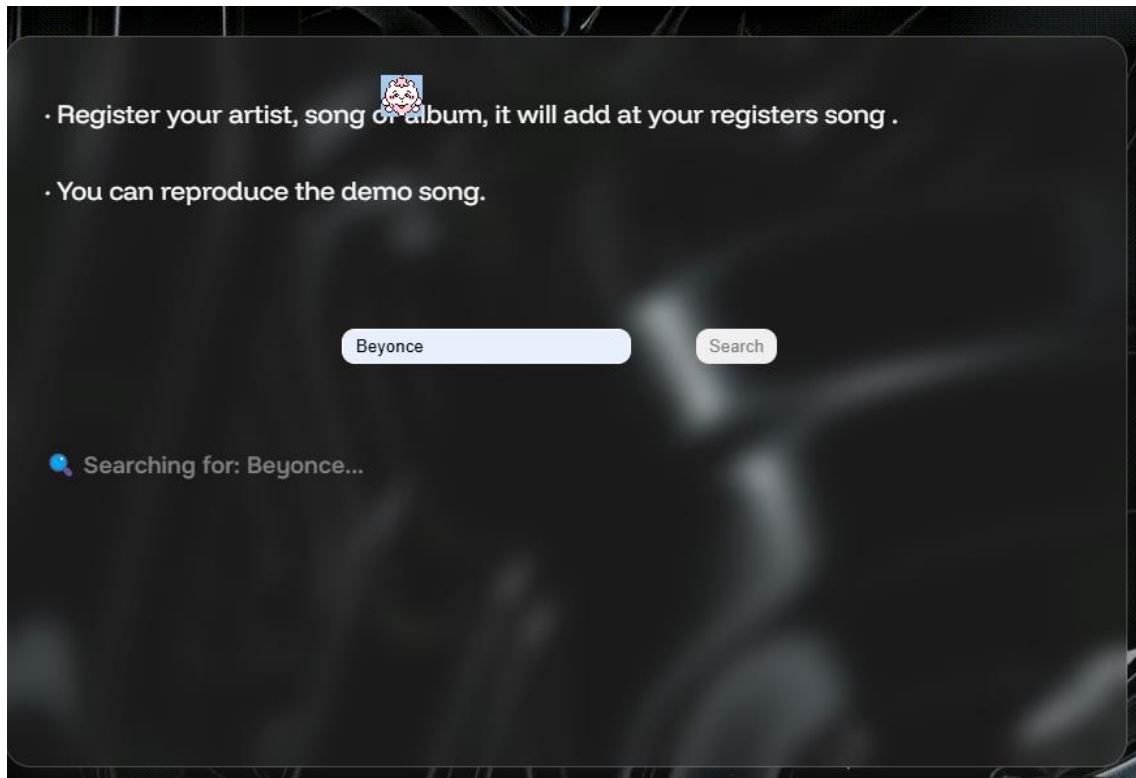


BIBLIOTECA DE CANCIONES

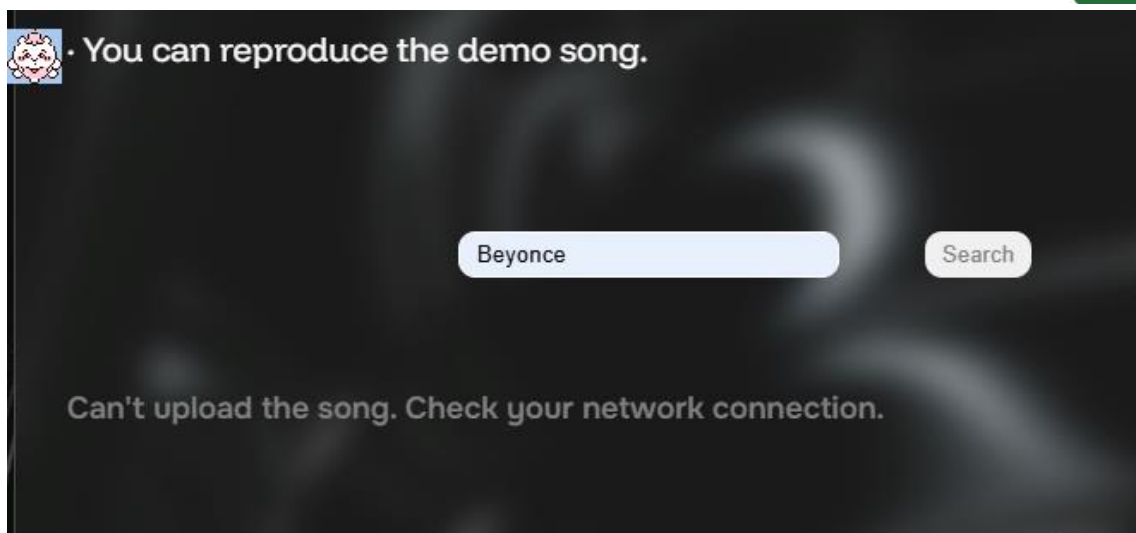
1.ENTREGA

Qué ventajas tiene mostrar un mensaje dinámico respecto a una presentación estática? Explica cómo lo hiciste en tu proyecto y en qué situación del mundo real puede aplicarse esta mejora. En mi proyecto lo he aplicado que si pulsase una canción en el buscador

¿Qué es el `textContent` en JavaScript y por qué lo usaste para mostrar el mensaje? Da un ejemplo con tu propio código. Coge un elemento del DOM del HTML y le incluye texto entre las etiquetas, como que obtiene el valor dentro de ellas. No lo he llegado a usar en mi web concretamente, he usado el `innerHTML` porque quiero generar yo dentro el contenido entre esas etiquetas de forma segura.



Maria



Maria

¿Qué problema evita la función `event.preventDefault()` en los formularios?

Explica su importancia con tus palabras y qué ocurriría si no la usaras. El `evento.preventDefault()` es evitar el movimiento que realiza una función en este caso lo he puesto para al picar en el botón de submit me salga un texto, con sus diferentes variantes usando un condicional.

¿Qué comprobación hiciste para evitar que el usuario dejara el campo vacío? Describe el código que usaste y qué mensaje de error mostraste. Muestra una captura del formulario básico y del mensaje personalizado funcionando.

Añade un breve comentario explicativo de la imagen. He puesto en el HTML

dentro del input el “required” ,

```
2 class="steps"> · You can reproduce the demo song.</h2>

form class="buscadorCanciones" id="formBusqueda">
  <input class="searchfafa" id="nombreBanda" type="text" placeholder="Write your song" required>
  <input class="submitbtn" type="submit" value="Search">
</form>
div class="cancionesDeezer" id="cancionesDeezer" aria-grabbed="true">
div>
ion>
```

¿Qué diferencias encuentras entre escribir un nombre en papel y hacerlo en un formulario digital? Reflexiona sobre la fiabilidad, velocidad y legibilidad en ambos sistemas. Pues a la hora de contemplar esos datos que se guarden y se verifiquen que pasen por algún proceso de corrección, y almacenamiento de datos del usuario.

¿Qué pasos del proceso tradicional (en papel) se han digitalizado en esta fase? Haz una lista breve y razona cada paso con un ejemplo del código.

¿Cómo afecta la digitalización a la accesibilidad de la información?

mejora la accesibilidad porque la información puede consultarse en cualquier momento, compartirse rápidamente y leerse fácilmente.

¿Crees que este formulario facilitaría el trabajo en una granja Pokémon real? ¿Qué función visual o técnica implementaste para mejorar la experiencia del usuario? Ejemplos: cambio de color, mensaje emergente, aviso de error, etc.

Si, porque agiliza mucho el proceso de introducción de datos y la búsqueda de esos.

Si otra persona quisiera reutilizar tu formulario, ¿qué partes del código necesitaría modificar? Especifica los fragmentos que se podrían adaptar fácilmente a otro contexto. Que cambiasen los placeholders a la hora de ponerle “nombre” al campo sin que pierda la forma de como lo tengo construido yo y luego cambiarle el form action para cambiarlo a otro servidor en el caso que se requiera el post de esos datos.

2. ENTREGA

¿Qué es una API y qué beneficios tiene utilizarla en una aplicación como esta?

Una API es como una interfaz que se comunica en dos programas entre si, que pueden pasar datos, cargarlos, etc. Aparte q no tienes que hacer un json de miles de cosas la sacas de una api. Las APIs tienen diferentes métodos nosotros solo hemos trabajado con GET y como mucho el POST.

2. ¿Qué diferencias hay entre usar .then() y async/await? ¿Cuál elegiste y por qué? El .then() es la reacción de la API, una vez que cargue el fetch pues convertirá eso en json , que es la respuesta que te devuelve el servidor, y como segundo hará lo otro, en este caso que me cojan las canciones y que me cree una serie de divs entre las etiquetas que yo solicito. El async await es un método mucho mas simplificado al realizar una promesa que de la manera usual.

3. ¿Cómo seleccionaste y mostraste los datos que devuelve la PokéAPI? Explica las propiedades que usaste (imagen, tipo, habilidades, etc.).

En mi caso use la API de Deezer para hacer la biblioteca de música. Al usar un .map en vez de un foreach, que me crea un nuevo array a partir del original y va rellenandolo cada vez que vea otra cancion, me busca los campos que tiene la api para que me lo rellene con la variable que he inicializado en el map. Como quiero presentar las canciones sin ningun detalle mas solo puse esos cuatro campos y ya. Tambien como quiero que me salgan las cuatro primeras pues he puesto el .slice() para que me cree un array pequeñito del original con solo esas 4 primeras posiciones.

```
function buscarCanciones(nombre) {  
  const url = `https://api.deezer.com/search?q=${encodeURIComponent(nombre)}`;  
  const resultado = document.getElementById("cancionesDeezer");  
  
  fetch(`https://corsproxy.io/?${url}`)  
    .then((response) => response.json())  
    .then((data) => {  
      const resultados = data.data;  
  
      if (resultados.length == 0) {  
        resultado.textContent = "🚫 No songs found";  
        return;  
      }  
      resultado.style.overflow = "scroll";  
      resultado.innerHTML = resultados  
        .slice(0, 4)  
        .map((song) => `  
        <div class="cancionSingular">  
            
          <h2>${song.title}</h2>  
          <h3>${song.artist.name}</h3>  
          <audio controls>  
            <source src="${song.preview}">  
          </audio>  
        </div>  
        `)  
        .join("");  
    })  
    .catch((error) => {  
      console.error("❌ Error al buscar la canción", error);  
    })  
}
```

4. ¿Qué estrategia utilizaste para gestionar errores si el nombre escrito no era válido?

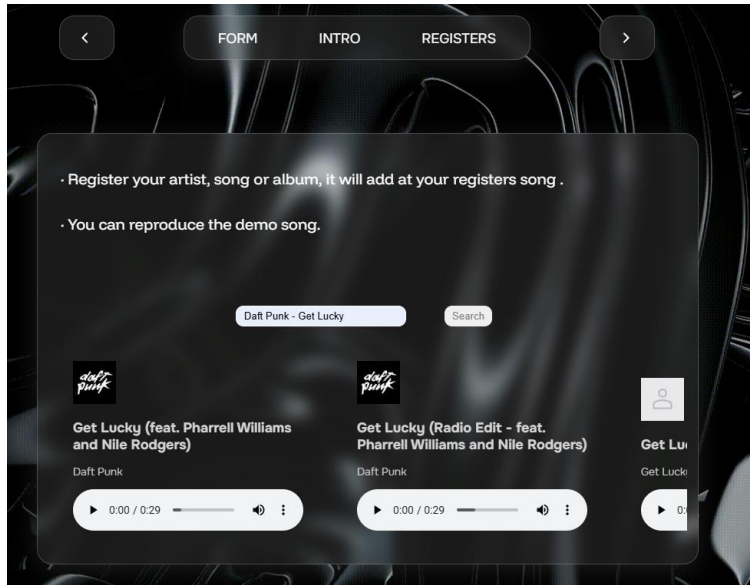
Pues puse un condicional que si la canción que me solicita el usuario no exista que me muestre un mensaje por pantalla, no confundir con el catch de error que

Maria Andreo

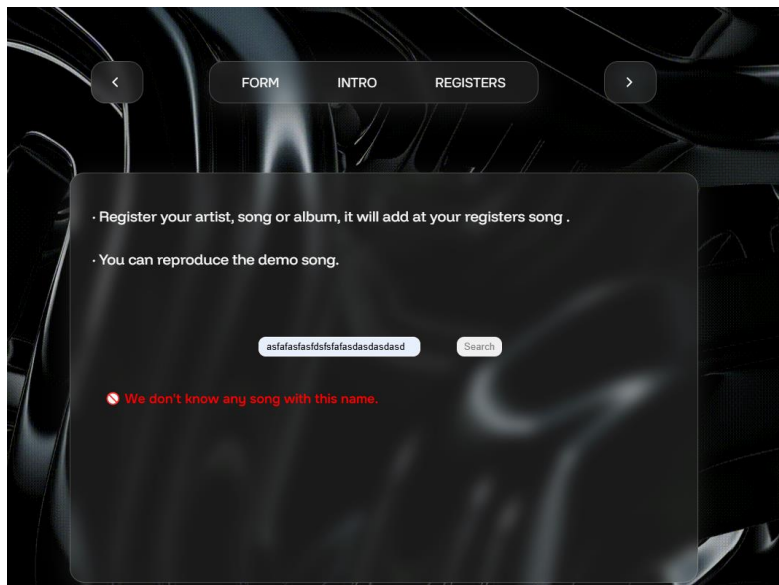
ese lo tengo para cuando la API no esta recibiendo bien las peticiones que se la hacen la pagina.

5. Muestra una captura de una búsqueda exitosa y una de un error gestionado correctamente. Añade una breve explicación.

Aquí es cuando funciona y la canción es encontrada.



Aquí es cuando el titulo de la canción no es correcto.



3. ENTREGA

1. ¿Qué función cumple localStorage en tu web? ¿Qué diferencia hay entre setItem y getItem?

Maria Andreo

En el localStorgae he guardado el nombre de la canción, artista o banda, el álbum y el audio preview. He hecho que al darle al play del audo salga una alerta de que se ha guardado en el localStorage del navegador para que posteriormente pueda usarse en el registro y hacer un filtraje de géneros de música etc..(en este punto no lo he hecho todavía pero es teoría)

```
<audio controls data-song='${JSON.stringify(song)}'> //id="audioControls"
  <source src='${song.preview}'>
</audio>
</div>
)
.join("");
//esto es cuando
const audios = resultado.querySelectorAll("audio");
audios.forEach((audio) => {
  audio.addEventListener("play", () => {
    const songData = JSON.parse(audio.getAttribute("data-song"));

    const cancionReproducida = {
      title: songData.title,
      artist: songData.artist.name,
      album: songData.album.title,
      preview: songData.preview,
      image: songData.album.cover_medium
    };

    // Guardamos en LocalStorage
    let cancionesGuardadas = JSON.parse(localStorage.getItem("cancionesReproducidas")) || [];
    cancionesGuardadas.push(cancionReproducida);
    localStorage.setItem("cancionesReproducidas", JSON.stringify(cancionesGuardadas));

    alert(" Song has been saved", cancionReproducida);
  });
});
}).catch((error) => {
  console.error("❌ Error al buscar la canción:");
});
```

Application

- Manifest
- Service workers
- Storage
 - Local storage
 - http://127.0.0.1:5500
 - Session storage
 - Extension storage

Key	Value
cancionesReproducidas	[{"title": "Instant Crush (feat. Julian Casablancas)", "artist": "Daft Punk", "album": "Random Access Memories", "preview": "https://cdnt-preview.dzcd..."}]

```
▶ [{"title": "Instant Crush (feat. Julian Casablancas)", "artist": "Daft Punk",,...}]
▶ 0: {"title": "Instant Crush (feat. Julian Casablancas)", "artist": "Daft Punk",,...}
▶ 1: {"title": "Faith", "artist": "Mateo", "album": "Revitalizing",,...}
▶ 2: {"title": "Bora Bora", "artist": "Mateo", "album": "Bora Bora",,...}
```

Con el get item lees el valor almacenado en el localStorage y se pueden acceder a los datos q hayas guardado. Con el set es que busca un valor dentro del localStorage con una clave en especifica por ejemplo cancionesReproducidas, es como vas a verlo dentro, se usa para guardar el valor en el localStorage, el otro se

obtiene en un valor que ya esta almacenado previamente. Osea primero viene el nombre y luego ya vienen los campos rellenos con los datos que carga de la api y lo lee en texto plano de JSON.

2. ¿Qué significa JSON.stringify() y por qué lo necesitas para guardar datos?

Puede convertir cualquier valor de javascript en un string, por ejemplo, objetos, arrays en strings del json.

3. ¿Cómo creaste y aplicaste el filtro por tipo o habilidad? ¿Qué función utilizaste? Hice un condicional, que si es el valor del tipoSeleccionado, que recoge el valor de cada opción que tengo en el HTML, coincida el de “todos los artistas” con el valor de “todos” que me salga el menú normal, y en el else es donde se aplica el filtro, que por ejemplo si “c” coincide con el genero, que se haga el div de ‘cancionSingular’, c, representa la canción si el usuario elige "rap" en el

menú desplegable, se quedarán solo las canciones donde c.tipo === "rap".

```
filtroTipo.addEventListener("change", () => {
  const tipoSeleccionado = filtroTipo.value;
  contenedorRegistros.innerHTML = " ";

  let cancionesFiltradas = [];
  // const filtradas = tipoSeleccionado === "todos"
  // ? canciones
  // : canciones.filter(c => c.tipo === tipoSeleccionado);
  //esto es como el if mas simplificado osea es una constante que siempre sera igual
  //el ? es como si es diferente a eso que tiposeleccionado sera igual a canciones y

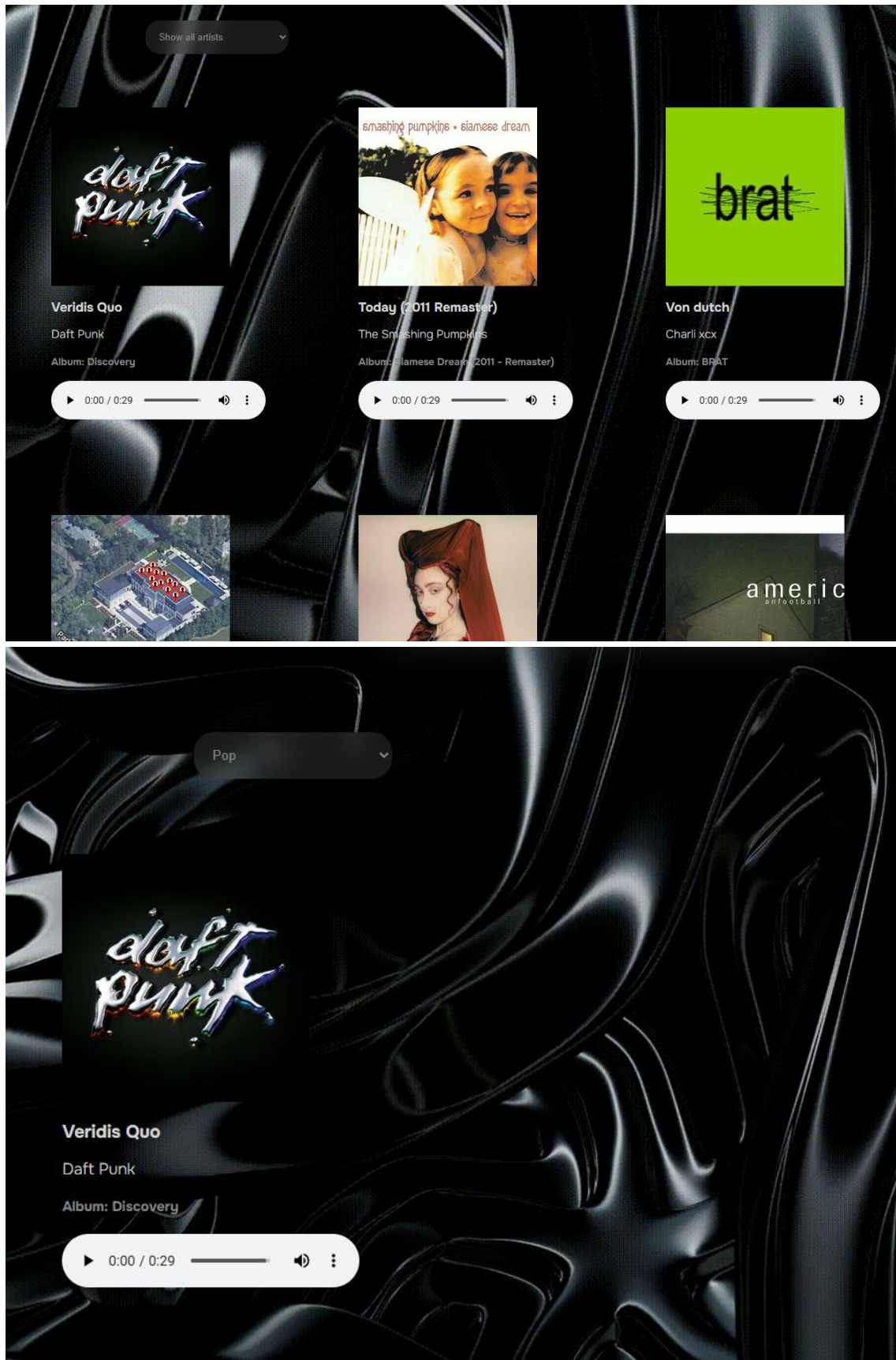
  if (tipoSeleccionado === "todos") {
    cancionesFiltradas = canciones;
    //esto es pa q se me vuelva a printar todas las canciones osea q si va a
    //jazz q te printe esas y luego cuando vuelva al general q es todos q se vuelva
    //printar porq antes no me lo hacia con esto si
    cancionesFiltradas.forEach(song => {
      contenedorRegistros.innerHTML += `
        <div class="cancionSingular">
          
          <h2>${song.title}</h2>
          <h3>${song.artist}</h3>
          <h4>Album: ${song.album}</h4>

          <audio controls>
            <source src="${song.preview}">
          </audio>
        </div>
      `;
    });
  } else {
    cancionesFiltradas = canciones.filter(c => c.tipo === tipoSeleccionado);
    cancionesFiltradas.forEach(song => {
      contenedorRegistros.innerHTML += `
        <div class="cancionSingular">
          
          <h2>${song.title}</h2>
          <h3>${song.artist}</h3>
          <h4>Album: ${song.album}</h4>
      `;
    });
  }
});
```

4. ¿Cómo dividiste tu interfaz en secciones (registro, búsqueda, reflexión)?

¿Qué método usaste? Yo lo dividí en tres htmls, uno que quedé constancia el registro al darle al play aparecerá en el html de registros. En el índice estará el buscador de las canciones y en el formulario aparecerá las canciones registradas de ese artista y/o banda, con los álbumes registrados, canciones y su género.

5. Muestra una captura de tu filtro funcionando y otra tras recargar la página con los datos restaurados.



6. ¿Qué ventajas notaste al dividir tu código en funciones? ¿Qué parte fue más fácil de mantener?

Maria Andreo

Al fraccionar todo por partes las funciones son más modulares.

7. Reflexiona sobre la diferencia entre registrar manualmente y tener este sistema digital. ¿Qué has automatizado?

El registro de respuesta de formularios y la búsqueda de verificación de estos. Una vez ya dado al botón “submit” esos datos ya han recibido constancia en el localStorage.

Mirar el proyecto en GitHub:

<https://github.com/mariamarM/DeezerLibraryAndreo/>